

SECTION 10 BUS OPERATION

This section describes the functional characteristics of the MC68881/MC68882 (FPCP) bus interface and the mechanisms used to execute data transfers between the FPCP and the main processor. This discussion includes descriptions of the functional characteristics of individual bus cycles as well as descriptions of the operand transfer protocols that require multiple bus cycles.

Although the FPCP is designed primarily for use as a coprocessor to the MC68020/MC68030 (MPU), there are no characteristics of the bus operation that preclude the use of the FPCP as a peripheral device with any other processor. This is because the M68000 Family coprocessor interface utilizes standard bus cycles to transfer instructions and data between the main processor and coprocessors in a system, with no special signals required for these transfers. Because of this general-purpose transfer mechanism, the type of the main processor and the nature of the system bus interface are transparent to the FPCP.

10.1 BASIC TRANSFER MECHANISM OVERVIEW

In order to execute a floating-point instruction, the FPCP and the main processor communicate using a series of bus cycles, instructions, and data according to a predefined protocol as described in **7.5 INSTRUCTION DIALOGS**. Most of these bus cycles transfer an entire item in a single transfer, although large items such as extended precision floating-point numbers require multiple bus cycles to transfer the entire operand. Also, if an FPCP port size of 8 or 16 bits is selected, multiple bus cycles can be required to transfer items that can be transferred with a single cycle over a 32-bit port.

10

The communications mechanism utilized by the FPCP and the main processor uses a set of mail-box registers, called the coprocessor interface registers (CIRs), to move data, instructions, and control information between the devices. The characteristics of the CIRs and the manner in which they are used by the FPCP and a main processor are described in **SECTION 7 COPROCESSOR INTERFACE**. The discussions in the following paragraphs are not specific to any particular CIR or instruction protocol, except where noted.

When a single bus cycle is able to accommodate an entire item, the transfer mechanism is obviously quite simple and the only requirement that must be met is that the bit alignment of the FPCP and main processor match. Figure 10-1 shows the bit assignment and significance of the 32-bit data bus of the FPCP, which must be matched to the main processor (for the MPU, this matching is accomplished by connecting D31 of the FPCP to D31 of the MC68020, D30 to D30, etc.).

When multiple bus cycles are required to transfer an item, the additional requirements of correct transfer order and port alignment must also be met. Figure 10-2 shows the data alignment of the FPCP for each port size. In this figure, if a section of the data bus is shaded for a particular encoding of SIZE, A4, A1 and A0, that section of the data bus is active during the transfer (i.e., valid data is expected during a write cycle, and the bus is driven

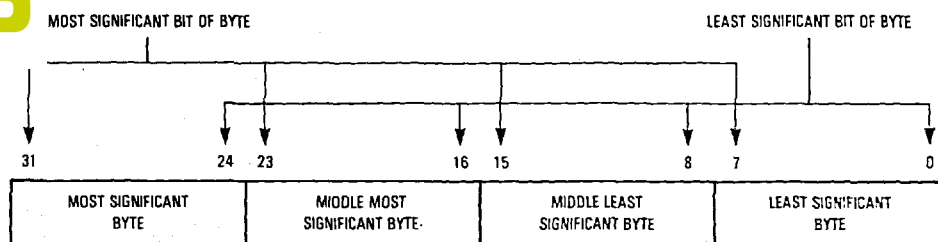


Figure 10-1. FPCP Data Bus Bit Assignments

SIZE	A4	A1	A0	PORT SIZE	DSACK1/DSACK0	ACTIVE DATA BUS SECTIONS
H	0	x	1	32 BITS	L H	
H	1	x	1		L L	
H	0	x	0	16 BITS	L H	
H	1	0	0		L H	
H	1	1	0		L H	
L	0	x	0	8 BITS	H L	
L	0	x	1		H L	
L	1	0	0		H L	
L	1	0	1		H L	
L	1	1	0		H L	
L	1	1	1		H L	

Figure 10-2. Data Bus Activity vs Port Size and Operand Alignment

during a read cycle). Otherwise, it is idle during the transfer. Note that the port size is not determined by the $\overline{\text{SIZE}}$ pin alone, but by the combination of the $\overline{\text{SIZE}}$ pin and A0. The following paragraphs describe the transfer order for each port size.

10.1.1 32-Bit Port Size

When $\overline{\text{SIZE}}$ and A0 are both high, the FPCP port size is defined to be 32 bits. In most cases, this configuration is statically selected by connecting the $\overline{\text{SIZE}}$ and A0 pins directly to VCC; although dynamic port size selection is possible if the proper timing constraints are followed for the $\overline{\text{SIZE}}$ and A0 pins. Although this configuration selects a 32-bit port size, the FPCP utilizes the dynamic bus sizing capabilities of the MPU to reduce the amount of multiplexing logic on the chip. The value of A4 during a bus cycle determines which bytes of the 32-bit port are used to drive or receive data. Since all of the coprocessor interface registers in the lower half of the CIR address range (A4=0, offsets \$00 through \$0F) are 16-bit registers, dynamic bus sizing is utilized to place all of those CIRs on data bus pins D32–D16, and the



K encoding returned indicates a 16-bit port size. All of the CIRs in the upper half of R address range ($A4 = 1$, offsets \$10–\$1F) are either 32-bit registers or 16-bit registers paired with undefined register locations. Therefore, the DSACK encoding used to terminate accesses in this range indicates a 32-bit port (during a read of the register select CIR, data bits 15–0 are undefined, reserved, and are driven high). In both of these cases, A4 determines the DSACK encoding that is returned, and A1 selects the appropriate word location. A0 is always one, to select a 32-bit FPCP port size, and thus individual bytes cannot be accessed in this configuration. Furthermore, the FPCP always expects a full 16 or 32 bits of data to be transferred during a bus cycle when $\overline{SIZE}$ is high; A0 is one, and A4 is zero or one, (with the exception of immediate byte or word operands, as discussed in the next paragraph).

When the FPCP is used in a 32-bit configuration, most CIR accesses transfer an entire instruction or data item in a single bus cycle. The one exception to this is for accesses to the operand CIR, which is used to transfer large items such as floating-point numbers and state frames. When an item is larger than four bytes, multiple accesses of the operand CIR are required to complete the transfer. In this case, the correct transfer order must be observed, in addition to the bit and byte alignment previously discussed. In all cases, each part of an item is transferred with the most significant bit aligned with bit 31 of the operand CIR (i.e., they are transferred across D31–D24, D31–D16, or D31–D0 for bytes, words, or long words, respectively). With the exception of byte and word immediate operands, the FPCP never requests the transfer of an item that is not a multiple of four bytes in length. An immediate byte or word operand is transferred in a single bus cycle and is left-aligned with the operand CIR. All other operands are transferred through the operand CIR in 32-bit units until the entire item is transferred.

When multiple bus cycles are required to transfer an item, the first operand CIR access transfers the most significant long word of the item; each successive access transfers the next least significant long word. For example, when an extended precision number is moved, the first operand CIR access is used to transfer bits 95–64 of the operand, the second access transfers bits 63–32, and the third access transfers bits 31–0 to complete the operand transfer. Note that the manner in which the operand is read from or written to memory is transparent to the FPCP, which allows the operand to be stored in memory in the native format of the main processor.

10

The amount of data transferred with each access to the operand CIR is dependent on the state of an instruction dialog and is determined by the FPCP, not the main processor. For example, if the FPCP issues an evaluate effective address and transfer data primitive with a length of 12 bytes, three accesses of the operand CIR are expected (with each access transferring four bytes). Thus, for a 32-bit port, the main processor is not allowed to transfer the operand with a series of word or byte transfers, but must use long-word transfers to move the operand.

10.1.2 16-Bit Port Size

When $\overline{SIZE}$ is high and A0 is low, the FPCP port size is defined to be 16 bits. In most cases, this configuration is statically selected by connecting the $\overline{SIZE}$ and A0 pins directly to VCC and GND, respectively, although dynamic port size selection is possible if the proper timing constraints are followed for the $\overline{SIZE}$ and A0 pins. Although $A0 = 0$ in this case, this value is not specifically used to select even byte addresses; rather, it is used to configure the data port to be 16 bits wide. When the FPCP is configured in this manner, all CIR accesses



assumed to transfer a full 16 bits to the word address selected by A1 (except for the case of an immediate byte operand, as discussed in a following paragraph). The DSACK encoding returned always indicates that the port is 16 bits wide; individual bytes cannot be accessed in this configuration.

In order to eliminate the need for on-chip multiplexing, the FPCP drives data on or receives data from only 16 bits of the data bus, depending on the encoding of A1 and A4 (thus allowing D31 and D15 of the FPCP to be tied together, D30 to be tied to D14, D29 to D13, etc., as described in **SECTION 11 INTERFACING METHODS**. For all accesses with A4 equal to zero, or with A4 equal to one and A1 equal to zero, data is transferred across D31–D16. Data is transferred across D15–D0 when A4 and A1 are both equal to one.

When the FPCP is used in the 16-bit configuration, most CIR accesses transfer an entire instruction or data item in a single bus cycle. The one exception to this is for accesses to the operand CIR, which is used to transfer large items such as floating-point numbers and state frames. When an item is larger than two bytes, multiple accesses of the operand CIR are required to complete the transfer. In this case, the correct transfer order must be observed, in addition to the bit and byte alignment previously discussed. In all cases, each part of an item is transferred with the most significant bit aligned with bit 31 or bit 15 of the operand CIR, depending on the value of A4 and A1 as described in the previous paragraph. With the exception of byte and word immediate operands, the FPCP never requests the transfer of an item that is not a multiple of four bytes in length. Immediate byte operands are transferred in a single bus cycle and are left-aligned with the operand CIR (i.e., they are transferred across D31–D24). All other operands are transferred through the operand CIR in 16-bit units until the entire item is transferred.

When multiple bus cycles are required to transfer an item, the first operand CIR access transfers the most significant word of the item; each successive access transfers the next least significant word. For example, when an extended precision number is moved, the first operand CIR access is used to transfer bits 95–80 of the operand, the second access transfers bits 79–64, and the third through sixth accesses transfer bits 63–48, 47–32, 31–16 and 15–0, respectively, to complete the operand transfer. Note that the manner in which the operand is read from or written to memory is transparent to the FPCP, which allows the operand to be stored in memory in the native format of the main processor.

The amount of data transferred with each access to the operand CIR is dependent on the state of an instruction dialog and is determined by the FPCP, not the main processor. For example, if the FPCP issues an evaluate effective address and transfer data primitive with a length of 12 bytes, six accesses of the operand CIR are expected (with each access transferring two bytes). Thus, for a 16-bit port, the main processor is not allowed to transfer the operand with a series of long-word or byte transfers, but must use word transfers to move the operand.

10.1.3 8-Bit Port Size

When the SIZE signal is low, the FPCP port size is defined to be eight bits. In most cases, this configuration is statically selected by connecting the SIZE pin directly to GND, although dynamic port size selection is possible if the proper timing constraints are followed for the SIZE and A0 pins. In this case, the value of A0 is used to select the correct byte address, rather than to configure the data port size. When the FPCP is configured in this manner, all CIR accesses transfer one byte to the address selected by A4–A0, and the DSACK



ling returned always indicates that the port is 8 bits wide. In order to eliminate the for on-chip multiplexing, the FPCP drives data on or receives data from only 8 bits of the data bus, depending on the encoding of A0, A1 and A4 (thus allowing D31, D23, D15 and D7 of the FPCP to be tied together; D30 to be tied to D22, D14 and D6; D29 to D21, D13 and D5, etc., as described in **SECTION 11 INTERFACING METHODS**). Figure 10-2 shows which bytes of the data bus are driven or received for each encoding of the A0, A1, and A4 lines.

When the FPCP is used in the 8-bit configuration, most transfers require multiple CIR transfers to move an entire instruction or data item. The one exception to this is for accesses to the operand CIR to transfer a byte immediate operand. When an item is larger than one byte, multiple accesses of the appropriate CIR are required to complete the transfer. In this case, the correct transfer order must be observed, in addition to the bit and byte alignment previously discussed. In all cases, each part of an item is transferred with the most significant bit aligned with bit 31, 23, 15, or 7 of the FPCP, depending on the value of A0, A1, and A4 as described in the previous paragraph. With the exception of byte and word immediate operands, the FPCP never requests the transfer of an item that is not a multiple of four bytes in length. Immediate byte operands are transferred in a single bus cycle and are left-aligned with the operand CIR (i.e., they are transferred across D31–D24). All other operands are transferred through the appropriate CIR in 8-bit units until the entire item is transferred.

When multiple bus cycles are required to transfer an item, the first operand CIR access transfers the most significant word of the item; each successive access transfers the next least significant word. For example, when an extended precision number is moved, the first operand CIR access is used to transfer bits 95–88 of the operand, the second access transfers bits 87–80, and the third through twelfth accesses transfer bits 79–72, 71–64, 63–56, 55–48, 47–40, 39–32, 31–24, 23–16, 15–8 and 7–0, respectively, to complete the operand transfer. Note that the manner in which the operand is read from or written to memory is transparent to the FPCP, which allows the operand to be stored in memory in the native format of the main processor.

The amount of data transferred with each access to the operand CIR is dependent on the state of an instruction dialog and is determined by the FPCP, not the main processor. For example, if the FPCP issues an evaluate effective address and transfer data primitive with a length of 12 bytes, 12 accesses of the operand CIR are expected (with each access transferring one byte). Thus, for an 8-bit port, the main processor is not allowed to transfer the operand with a series of word or long-word transfers, but must use byte transfers to move the operand.

10.2 RESET OPERATION

Before the FPCP can be used for any operation after power has been applied to the system, it must be initialized using a hardware reset function. This is done when power is initially applied to the system by asserting **RESET** for at least four clock cycles (with reference to the FPCP CLK signal) after VCC has reached the nominal operating level. After power has been stable and the FPCP has executed a power-up reset operation, a subsequent reset operation may be initiated by asserting **RESET** for at least two cycles of the FPCP CLK signal. Note that in order to maintain compatibility with all M68000 Family devices, the power-on reset pulse for a system should be a minimum of 100 ms, while a 10 clock minimum (with respect to the clock signal of the slowest M68000 Family device in the system) should be used for reset operations after power is stable.

When a hardware reset operation is performed, the FPCP immediately aborts any operation that may have been in progress and returns to the idle state. All of the floating-point data registers are loaded with nonsignaling NaNs, and the FPCR and FPSR are cleared to all zeros (thus clearing any old status information and selecting the IEEE standard default operating modes). An identical operation may be performed under software control by a FRESTORE of a null state frame (although a hardware reset must be executed at power-up in order to initialize the FPCP).

One consideration that should be given to the $\overline{\text{RESET}}$ signal of the FPCP is the treatment of a RESET instruction by an M68000 Family processor. When the RESET instruction is executed by an M68000 Family processor, the internal state of the processor is not affected, but the external system should respond to the reset operation. Since the FPCP is considered to be part of the internal state of the main processor, prudent system design suggests that the FPCP should not respond to the assertion of the $\overline{\text{RESET}}$ signal by the main processor. This can be accomplished in many ways, depending on the requirements of the system. A simple circuit to support this operation is shown in Figure 10-3. If a software RESET function is needed, it is suggested that this be implemented by an FRESTORE of a null frame.

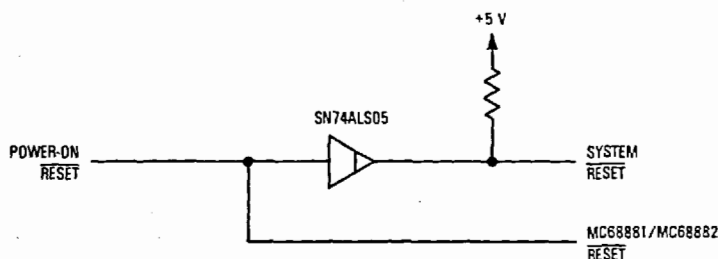


Figure 10-3. FPCP Reset Logic Example

10.3 CHIP SELECT TIMING

Most of the bus cycle timing requirements of the FPCP are straightforward, with all signal timing following the normal M68000 Family conventions. The only signal timing that is specific to the FPCP bus interface is the relationship of the assertion of chip select ($\overline{\text{CS}}$) to the assertion of the address strobe ($\overline{\text{AS}}$) and data strobe ($\overline{\text{DS}}$). Unlike most M68000 Family peripherals that require the assertion of $\overline{\text{CS}}$ to follow the assertion of $\overline{\text{AS}}$ or $\overline{\text{DS}}$, the FPCP allows the $\overline{\text{CS}}$ assertion to precede the assertion of the $\overline{\text{AS}}$ and $\overline{\text{DS}}$.

In order to detect the start or end of an access, the FPCP monitors the $\overline{\text{AS}}$, $\overline{\text{DS}}$, $\overline{\text{CS}}$, and $\text{R}/\overline{\text{W}}$ signals. A cycle start is detected when $\overline{\text{AS}}$, $\overline{\text{CS}}$, and $\overline{\text{DS}}$ or $\text{R}/\overline{\text{W}}$ (for a write cycle) are asserted, and a cycle end is detected when the first strobe ($\overline{\text{AS}}$ or $\overline{\text{DS}}$) is negated. The order in which these signals are sequenced is not critical to correct operation, and in the case of $\overline{\text{CS}}$ the occurrence of a negated or asserted edge is not needed to detect a new access. For example, it is not required that $\overline{\text{CS}}$ be negated between successive accesses to the FPCP, since the negation and assertion of the $\overline{\text{AS}}$ and $\overline{\text{DS}}$ signals causes the FPCP to detect the end of one access and the start of the next.

The FPCP conditions the $\overline{\text{DSACK}}$ generation logic internally with $\overline{\text{AS}}$ and $\overline{\text{CS}}$. To ensure that $\overline{\text{DSACK}}$ assertion is not delayed longer than necessary, $\overline{\text{CS}}$ should be asserted before



asserted (since $\overline{CS}$ is system dependent but $\overline{AS}$ is MPU dependent). This design is "early chip select". On the other hand, when $\overline{CS}$ is asserted after $\overline{AS}$ has been asserted, the design is called "late chip select". A late chip-select design may add wait states to the FPCP accesses.

A timing restriction on $\overline{CS}$ occurs on a FPCP access followed immediately by a non-FPCP access. $\overline{CS}$, which is asserted during the FPCP access, must negate in time for it to occur before the assertion of $\overline{AS}$ of the subsequent non-FPCP access.

To satisfy this timing restriction with an early chip select, neither $\overline{AS}$ or $\overline{DS}$ can be used to generate $\overline{CS}$. Figure 10-4 shows some circuits that correctly generate an early chip-select signal for MPU-based systems. Note that in these circuits only the following terms are included in the $\overline{CS}$ equation:

- FC2-FC0=7 — CPU Space
- A19-A16=2 — Coprocessor Communications
- A15-A13=1 — Cp-ID One (Freescale Assembler Default)

For systems that use the MC68020 or the MC68030 with an FPCP, the maximum time for an early chip select is:

$$t_{AVCS} = t_{AVSA}$$

where:

t_{AVCS} = Address/function code valid to $\overline{CS}$ asserted (maximum).

t_{AVSA} = MC68020/MC68030 address/function code valid to $\overline{AS}$ asserted (AC electrical specification #11 minimum).

For a 20-MHz MPU and a 25-MHz FPCP:

$t_{AVCS} = 10$ ns maximum.

The 74AS02 and 74AS30 implementation shown in Figure 10-4 or a PAL implementation with a maximum decode delay time of 10 ns may be used.

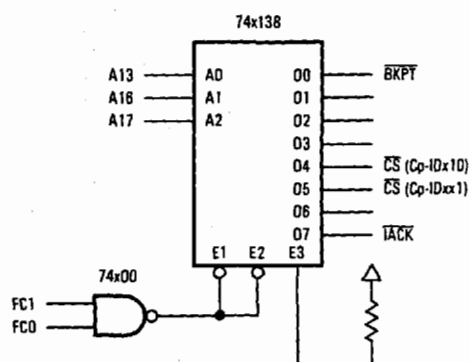
For a 25-MHz and 33-MHz MPU/FPCP system, refer to EB116 entitled *Chip-Select Generation for a 33.33-MHz MC68030 Microprocessor and a 33.33-MHz MC68882 Floating-Point Co-processor*.

A late chip-select design can use slower (and, therefore, less expensive) logic. To implement this design, the $\overline{CS}$ generation logic should include $\overline{AS}$. Another consideration in using slower logic is that if a non-FPCP access follows an FPCP access, the $\overline{CS}$ for the FPCP must not remain asserted inadvertently during the non-FPCP access. However, $\overline{AS}$ should be included in the decode logic as shown in Figure 10-5(A), along with the timing that the logic provides. When $\overline{AS}$ is used in an AND gate with the decode logic output as shown Figure 10-5(B), $\overline{CS}$ is asserted after the start of the non-FPCP access, as the timing diagram shows. This implementation in Figure 10-5(B) is incorrect.

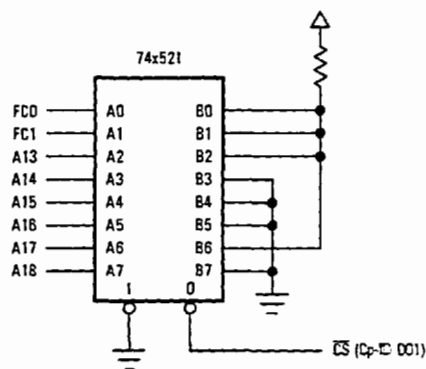
10.4 BUS CYCLE FUNCTIONAL DESCRIPTIONS

The FPCP executes three types of bus cycles, according to the direction of the transfer and the CIR that is selected by the main processor. The three bus cycle types are: synchronous read cycles, asynchronous read cycles, and asynchronous write cycles. In this context, the terms synchronous and asynchronous convey slightly different meanings than when they

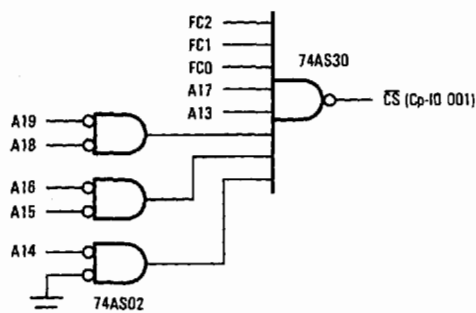
FC0-FC2 — \$7
CPU Space (A16-A19) — \$2
Cp-ID (A13-A15) — \$1 (default)



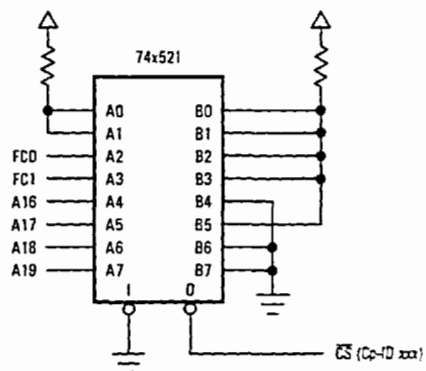
Up to two coprocessors in the system, with additional decode for BKPT and IACK cycles.



Up to seven coprocessors in the system.



Up to seven coprocessors in the system.



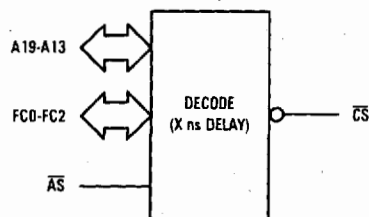
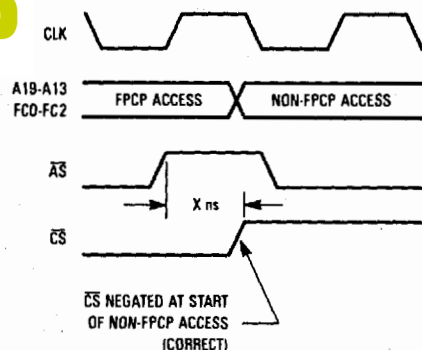
Only one coprocessor in the system.

Decode Delay Times*

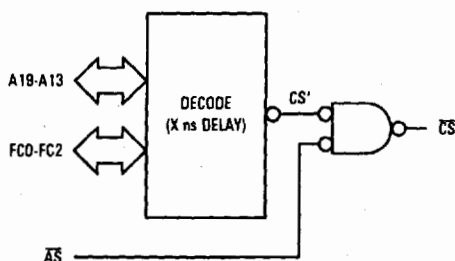
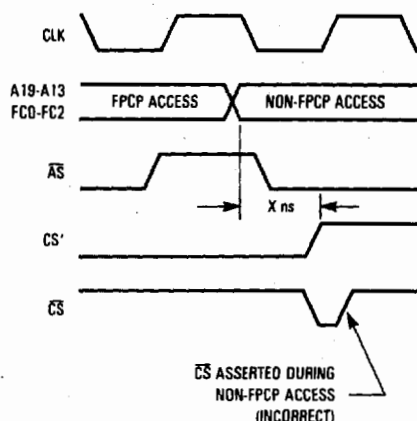
Programmable Array Logic (PAL)	= 10 - 25 ns max.
74x00/74x138 combined propagation delay	= 9.5 - 12.8 ns max.
74x521 compare delay	= 5.5 - 11 ns max.
74AS02/74AS30 combined propagation delay	= 9 ns max.

*The 'x' represents various combinations of logic families including F, AS, or fast CMOS.

Figure 10-4. Example of Early Chip Select Circuits



a) CORRECT Implementation of a "late chip select"



b) INCORRECT Implementation of a "late chip select"

Figure 10-5. Example of Late Chip-Select Circuit

are used to describe the bus transfer characteristics of a microprocessor (e.g., where the MC68030 microprocessor and an external device perform either synchronous or asynchronous cycles, depending on the relationship between the clocks used in the MPU and the external device). Here, the terms synchronous and asynchronous are used with respect to the FPCP clock. The following paragraphs describe the functional characteristics of each bus cycle type (for an AC parametric description of the FPCP bus interface, refer to **SECTION 12 ELECTRICAL SPECIFICATIONS**).

In the following discussions, the main processor is assumed to be an MC68020 or MC68030, with the FPCP and the MPU both driven by the same clock signal. Thus, the terminology and conventions used are identical to the bus description for the MPU. This clock frequency relationship is not required, but the following discussions are simplified by assuming that both devices use the same clock signal. Where appropriate, references are made to variations in bus cycle operations if the main processor is not an MC68020 or MC68030.

10.4.1 Synchronous Read Cycles

When the main processor performs a read access to either the response or save CIR, the FPCP responds by executing a synchronous read bus cycle. In this context, the term synchronous signifies that the bus cycle timing is directly related to the FPCP clock signal, but

FPCP clock is not required to be synchronous with the main processor clock during transfer. By synchronizing the bus cycle to the FPCP clock, the appropriate response primitive or format word is always returned based on the current status of the FPCP. Also, since these bus cycles are used to transmit service requests to the main processor, the synchronous bus cycle timing allows the main processor and FPCP to be synchronized at critical points in an instruction dialog, without requiring synchronous clock signals for the two devices.

The functional timing for the synchronous read cycle is shown in Figure 10-6. The FPCP detects the start of a synchronous read cycle when chip select and address strobe are asserted, read/write is high, and the address pins are encoded to \$00 (to select the response CIR) or \$04 (to select the save CIR). When either of these conditions is met, the FPCP begins to sample the address strobe, data strobe, and chip-select lines on each rising edge of the CLK signal. When all three of these signals are sampled as asserted, the FPCP latches certain internal state flags and uses those flags to determine the appropriate response primitive or format word to be placed on the data bus. One and one-half clock cycles later, the FPCP begins to drive the data value onto the bus and assert the appropriate data transfer and size acknowledge encoding. The data value remains on the data pins and $\overline{DSACKx}$ remains asserted until the first of the two signals, $\overline{AS}$ or $\overline{DS}$, is negated; then the bus cycle is terminated by placing the data bus in the high-impedance state and negating $\overline{DSACKx}$.

As shown in Figure 10-6, this type of bus cycle requires five clock cycles (two wait cycles) when the MPU and the FPCP share the same clock. Under certain conditions, these bus

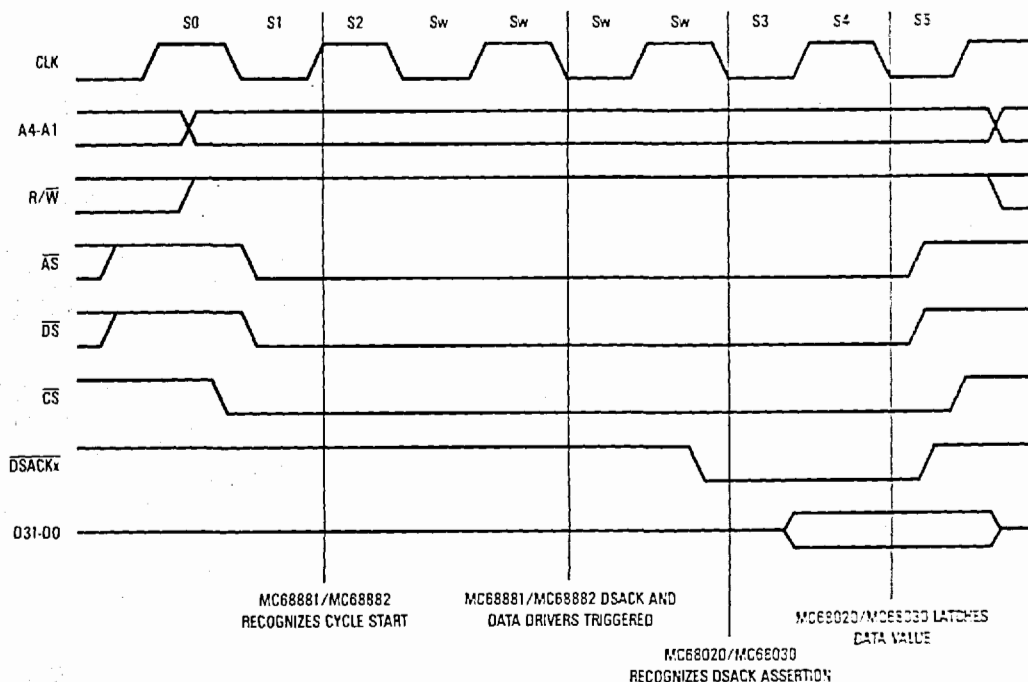


Figure 10-6. Synchronous Read-Cycle Timing Diagram



as may require as many as six or seven clock cycles. Two separate mechanisms define whether additional clock cycles are required for this type of a bus cycle:

1. The relationship between the assertion of $\overline{AS}$, $\overline{DS}$, or $\overline{CS}$ and the rising edge of the FPCP clock signal
2. The relationship between the assertion of $\overline{DSACKx}$ by the FPCP and the falling edge of the MPU clock signal

As previously described, $\overline{DSACKx}$ is triggered to assert one and one-half clock cycles after $\overline{AS}$, $\overline{DS}$, and $\overline{CS}$ are sampled as asserted; thus, the best-case timing occurs when all three of these signals are asserted as early in the bus cycle as possible. Since the MPU triggers the assertion of $\overline{AS}$ and $\overline{DS}$ with the falling edge of the CLK signal (which is assumed to be the same for both devices) and the FPCP samples those signals, along with $\overline{CS}$, on the rising edge of the CLK signal, the best-case cycle timing occurs only if $\overline{AS}$, $\overline{DS}$, and $\overline{CS}$ are all asserted to provide the required setup time to the next rising edge of the clock. Thus, the maximum assertion and propagation delays for these signals must be less than the clock pulse width low in order to guarantee the best-case bus cycle timing. Although the maximum specifications for the assertion, by the MPU, of $\overline{AS}$ or $\overline{DS}$ from the falling edge of the clock do not guarantee the best-case timing for operation at 16.67 MHz under worst-case system environments, the best-case timing normally occurs under typical system conditions. In order to assure the possibility that the best-case timing occurs, system designers should utilize the $\overline{CS}$ generation methods described in **10.3 CHIP SELECT TIMING** to prevent propagation delays of the $\overline{CS}$ logic from lengthening the bus cycle by one clock.

In the same manner as just described (where the FPCP misses the assertion of $\overline{AS}$, $\overline{DS}$, or $\overline{CS}$), one clock cycle may be added to the bus cycle timing if the MPU misses the assertion of $\overline{DSACKx}$ by the FPCP. The assertion of $\overline{DSACKx}$ by the FPCP is triggered by the falling edge of the clock, and the propagation delay for this assertion can be quite long (slightly longer than one 16.67 MHz clock cycle under worst-case system conditions). Since the MPU samples $\overline{DSACKx}$ on the falling edge of the clock, the assertion of $\overline{DSACKx}$ triggered by a given falling clock edge may not be completed ahead of the setup time to the next falling clock edge. There is very little that a system designer can do to assure that the $\overline{DSACKx}$ assertion is recognized on the first falling clock edge after it is triggered, since the propagation delay is dependent on individual device characteristics as well as system conditions such as temperature and power supply levels.

10

Due to the nature of the two mechanisms just described, it is possible that for an individual system the bus cycle timing for synchronous read cycles may be different under varying system conditions. For example, when a system is first turned on (and thus the devices are at room temperature) it is quite likely that synchronous read cycles require five clock cycles as shown in Figure 10-6. As the temperature increases to the normal operating range, the synchronous read cycle timing may change to six clock cycles. If the temperature rise affects both of the synchronization mechanisms enough (particularly if the $\overline{CS}$ generation logic causes the assertion of $\overline{CS}$ to follow the assertion of $\overline{AS}$ and/or $\overline{DS}$), the timing for these operations may increase to seven clock cycles or even vary on a cycle-by-cycle basis between six and seven clock cycles. Some other factors that may affect the timing for synchronous reads are the power supply levels for the FPCP and MPU, the individual device characteristics (due to manufacturing variances), and the capacitive loading of the control signals.

It should be noted that the timing variances for synchronous read cycles do not affect the overall performance of a system significantly. Specifically, one or two additional clock

s per synchronous read cycle results in a small percentage change in the overall execution time for an instruction (since most instructions typically require over 50 clock cycles to execute). The only environment where these timing variances may be of concern is when a programmer is attempting to optimize an instruction sequence for maximum overlap. In this case, these factors should be added to the instruction execution timing variability mechanisms discussed in **SECTION 8 INSTRUCTION EXECUTION TIMING**.

10.4.2 Asynchronous Read Cycles

When the main processor performs any access to the FPCP with $R/\overline{W}$ high other than a read of the response or save CIR, the FPCP responds by executing an asynchronous read cycle. In this context, the term asynchronous signifies that the bus cycle timing is not related to the FPCP or MPU clock signals in any way. The FPCP supports this type of operation by implementing all of the CIRs, except the response and save CIRs, as dual ported structures. Thus, the main processor can access these CIRs at the maximum speed regardless of the clock frequency of the FPCP, while the FPCP internally accesses these CIRs in a synchronous manner.

The functional timing for the asynchronous read cycle is shown in Figure 10-7. The FPCP detects the start of an asynchronous read cycle when chip select, address strobe, and data strobe are asserted; read/write is high; and the address pins are not encoded to S00 or S04 (which selects the response or save CIR, respectively). When this condition is met, the FPCP responds by placing the data from the selected CIR on the data bus and asserting the appropriate data transfer and size acknowledge encoding. The data value remains on the data pins and $\overline{DSACKx}$ remains asserted until the first of the two signals, $\overline{AS}$ or $\overline{DS}$, is negated at which time the bus cycle is terminated by placing the data bus in the high-impedance state and negating $\overline{DSACKx}$.

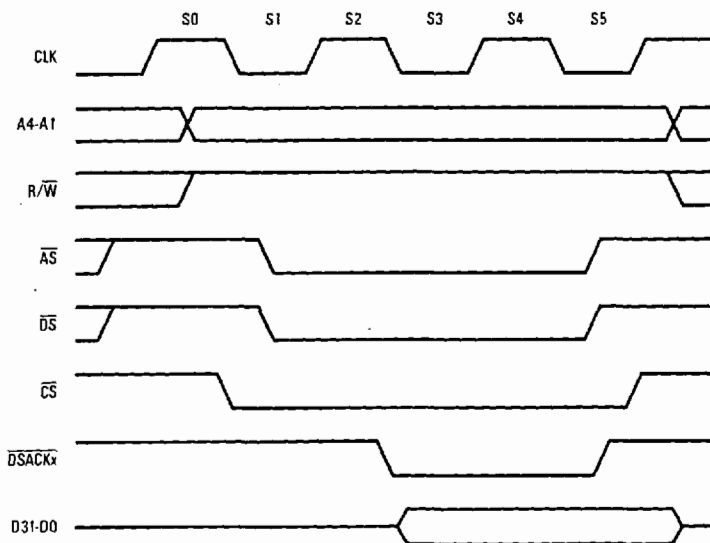


Figure 10-7. Asynchronous Read Cycle Timing Diagram

own in Figure 10-7, this type of bus cycle requires three clock cycles (no wait cycles) when the MPU and the FPCP share the same clock. Due to the asynchronous timing of the data transfer and size acknowledge assertion by the FPCP, this bus cycle timing does not depend on the clock frequency of the FPCP (there are some exceptions to this rule, as discussed in **10.5 INTER-CYCLE TIMING RESTRICTIONS**). For example, if the MC68881 clock frequency is 12.5 MHz and the MPU clock frequency is 16.67 MHz, this bus cycle requires three MPU clock cycles since the assertion of $\overline{DSACKx}$ is recognized by the MPU on the falling edge of S2. This assumes that the chip-select logic causes the assertion of $\overline{AS}$ and $\overline{DS}$ so that the $\overline{AS}/\overline{DS}$ assertion delay is not lengthened by the chip-select logic propagation time.

10.4.3 Asynchronous Write Cycles

When the main processor performs any access to the FPCP with $R/\overline{W}$ low, the FPCP responds by executing an asynchronous write cycle. The definition of asynchronous in the first paragraph of the preceding subsection applies also to asynchronous write cycles.

The functional timing for the asynchronous write cycle is shown in Figure 10-8. The FPCP detects the start of an asynchronous write cycle when chip select and address strobe are asserted and read/write is low. When this condition is met and an asserted pulse occurs on $\overline{DS}$, the FPCP responds by asserting the appropriate data transfer and size acknowledge encoding and latching the value of the data bus into the selected CIR. The $\overline{DSACKx}$ encoding remains asserted until $\overline{AS}$ is negated; then the bus cycle is terminated by negating $\overline{DSACKx}$.

As shown in Figure 10-8, this type of bus cycle requires three clock cycles (no wait cycles) when the MPU and the FPCP share the same clock. Due to the asynchronous timing of the data transfer and size acknowledge assertion by the FPCP, this bus cycle timing does not depend on the clock frequency of the FPCP (there are some exceptions to this rule, as

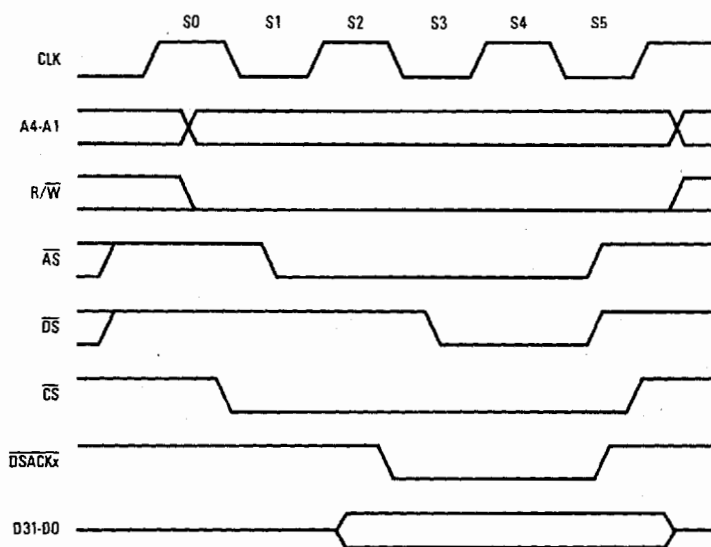


Figure 10-8. Asynchronous Write Cycle Timing Diagram



issued in **10.5 INTER-CYCLE TIMING RESTRICTIONS**). For example, if the MC68881 clock frequency is 12.5 MHz and the MPU clock frequency is 16.67 MHz, this bus cycle requires three MPU clock cycles since the assertion of $\overline{DSACKx}$ is recognized by the MPU on the falling edge of S2. This assumes that the chip-select logic causes the assertion of $\overline{CS}$ to precede the assertion of $\overline{AS}$ and $\overline{DS}$ so that the $\overline{AS}/\overline{DS}$ assertion to $\overline{DSACKx}$ assertion delay is not lengthened by the chip-select logic propagation time.

10.5 INTER-CYCLE TIMING RESTRICTIONS

The bus interface of the MC68881 is designed to operate satisfactorily at any reasonable clock frequency relationship between the MC68881 and the main processor. In most cases, differences in the clock frequency of the two devices does not affect the operation of the bus; and particularly, it does not affect the timing of individual bus cycles. However, there are some cases where the timing of a bus cycle is modified if the MC68881 is overrun by the main processor.

During coprocessor interface dialogs, certain bus cycles trigger actions by the FPCP on the negated edge of data strobe. Operations internal to the FPCP that are initiated in this manner are completed within four clock cycles after the negation of $\overline{DS}$, but the main processor may initiate a subsequent asynchronous bus cycle before those internal operations are completed. In these cases, the MC68881 delays the subsequent asynchronous access by not responding to the bus cycle (and thus not asserting $\overline{DSACKx}$) until the internal operations are completed. Synchronous accesses (i.e., accesses to the response or save CIR) execute in the normal manner regardless of preceding accesses. The following is a list of the bus cycles that initiate internal operations on the negated edge of $\overline{DS}$, where a subsequent asynchronous bus cycle might overrun the FPCP and necessitate a delay in the assertion of $\overline{DSACKx}$:

10

1. A write cycle to the least significant byte of the control CIR
2. A write cycle to the least significant byte of the restore CIR
3. The last write cycle to the least significant byte of the operand CIR during a restore operation with a busy state frame
4. The first read from the least significant byte of the operand CIR during a save operation with an idle or busy state frame

In all of these cases, the term **least significant byte** indicates a transfer of any size that includes the least significant byte of the referenced CIR and does not indicate that only byte transfers cause conditions that require delays in subsequent bus cycles.

In addition to the cases just described, the possibility exists that the main processor may overrun the FPCP if the main processor clock frequency is greater than that of the FPCP. There are two cases where this might occur:

1. The main processor reads the operand or register select CIR before the FPCP has data ready for transfer to the main processor.
2. The main processor writes to the operand CIR before the data from the previous write cycle has been stored internally.

In both of these cases, the FPCP does not respond to the initiation of an asynchronous bus cycle until the internal data transfers are completed (synchronous bus cycles are not delayed).

As just described, the FPCP delays asynchronous bus cycles, if necessary, until internal operations are completed. However, even though the response to these bus cycles is delayed, the FPCP bus interface unit control logic does detect the beginning of each access regardless of the state of the execution unit. Thus, it is possible that an access to a CIR may be detected before the bus interface unit has completed previous operations and updated status flags to reflect the state of an instruction dialog. This can result in spurious protocol violations if the coprocessor interface protocol is not strictly observed.

The most important protocol that must be observed is that the come-again request included by the FPCP in every evaluate effective address and transfer data primitive must not be ignored by the main processor. For example, if the come-again request is ignored and the main processor clock is much faster than the FPCP clock, the following situation might occur:

1. The main processor receives the evaluate effective address and transfer data request primitive, processes it, and begins to transfer the operand.
2. The last operand part is written to the operand CIR.
3. The main processor ignores the come-again request and begins execution of the next instruction immediately.
4. The next instruction is an FPCP instruction that the main processor initiates by writing the command word to the command CIR.
5. Since the internal operand transfer is not complete, the BIU flags still indicate that the next expected access is to the operand CIR; thus the access to the command CIR is deemed illegal, and a protocol violation occurs.

In this case, if the main processor follows the protocol and services the come-again request by reading the response CIR immediately after the last operand CIR access, a null (CA = 1, IA = 1) primitive may be returned by the MC68881. Since the response CIR read-cycle timing is synchronous with the MC68881 clock signal, this read cycle allows the main processor to be synchronized to the MC68881 internal operations. Thus, the next read of the response CIR normally occurs after internal operations are completed. At that time, the response encoding is changed to null (CA = 0) to allow the main processor to proceed, and the subsequent access to the command CIR is a legal access.

In addition to the previously mentioned restrictions, the MC68882 may return an evaluate <ea> and transfer data primitive with CA = 0, which does not require the main processor to read the response register before proceeding to the next instruction. After the last write to the operand CIR, if the next instruction is another MC68882 instruction, the write to the command CIR can occur immediately without adverse effects. However, if the read of the response CIR (which normally follows the write of the command CIR) occurs sooner than three MC68882 clocks after the completion of the previous operand CIR write operation, a protocol violation occurs. Therefore, if the main processor is an MC68020 or MC68030, its clock frequency cannot be more than 1.5 times the frequency of the MC68882 clock. Otherwise, the read response CIR operation might occur too soon and cause a protocol violation. A main processor other than an MC68020 or MC68030 must also observe this timing requirement to avoid a protocol violation.

